

Задание 1

Порядки элементов поля $\text{GF}(2^5)$

Мультипликативная группа поля $\text{GF}(2^5)$ является циклической группой порядка 24 и порождается примитивным элементом x . Элементы поля имеют вид $\{0\} \cup \{x^0, x^1, \dots, x^{23}\}$.

Порядок элемента 0 в мультипликативной группе не определён. Для ненулевого элемента x^k порядок вычисляется по формуле:

$$\text{ord}(x^k) = \frac{24}{\gcd(k, 24)}.$$

Таблица порядков элементов:

Элемент	k	Порядок
$1 = x^0$	0	1
x^1	1	24
x^2	2	12
x^3	3	8
x^4	4	6
x^5	5	24
x^6	6	4
x^7	7	24
x^8	8	3
x^9	9	8
x^{10}	10	12
x^{11}	11	24
x^{12}	12	2
x^{13}	13	24
x^{14}	14	12
x^{15}	15	8
x^{16}	16	3
x^{17}	17	24
x^{18}	18	4
x^{19}	19	24
x^{20}	20	6
x^{21}	21	8
x^{22}	22	12
x^{23}	23	24

Для вычисления порядков всех элементов использовался следующий Python код:

```
import math

for k in range(24):
```

```
order = 24 // math.gcd(k, 24)
print(f"{k:2d} | {order}")
```

Задача 2

Циклический (15, 4)-код и LFSR

Рассматривается циклический (15, 4)-код над полем $\mathbb{F}_2$ с проверочным многочленом $h(x) = 1 + x + x^4$. Кодер реализован на основе регистра сдвига с линейной обратной связью (LFSR) длины 4, соответствующего полиному $h(x)$. Начальное заполнение регистра: 0011.

Смена состояний регистра:

0011 $\rightarrow$ 1000 $\rightarrow$ 0100 $\rightarrow$ 0010 $\rightarrow$ 0001 $\rightarrow$ 1001 $\rightarrow$
 1101 $\rightarrow$ 1111 $\rightarrow$ 1110 $\rightarrow$ 0111 $\rightarrow$ 1010 $\rightarrow$ 0101 $\rightarrow$ 1011 $\rightarrow$ 1100 $\rightarrow$ 0110 $\rightarrow$
 0011

Регистр прошёл через все 15 ненулевых состояний и вернулся в исходное состояние. Таким образом, период равен $15 = 2^4 - 1$, и код является кодом максимальной длины.

Минимальное расстояние кода $d_{\min} = 8$. Данный код представляет собой симплекс-код (15, 4, 8) — дуальный коду Хэмминга (15, 11, 3). Все ненулевые кодовые слова имеют вес $8 = 2^4 - 1$.

Для симуляции работы регистра, проверки периода и вычисления минимального расстояния использовался следующий Python-код:

```
def simulate_lfsr(initial=[0,0,1,1]):
    state = initial[:]
    sequence = []
    seen = set()
    for _ in range(20):
        seq_str = ''.join(map(str, state))
        sequence.append(seq_str)
        tup = tuple(state)
        if tup in seen:
            break
        seen.add(tup)

        fb = state[0] ^ state[3]
        # сдвиг влево
        state = state[1:] + [fb]

    print("Состояния:", ' -> '.join(sequence[:16]))
    print("Период:", len(seen))
    return len(seen) == 15

simulate_lfsr()
```

```

# Вычисление d_min (перебор всех 16 кодовых слов)
import numpy as np
G = np.array([
    [1,0,0,0, 1,0,0,1,0,1,1,1,0,1,0],
    [0,1,0,0, 0,1,0,0,1,0,1,1,1,0,1],
    [0,0,1,0, 1,0,1,0,0,1,0,1,1,1,0],
    [0,0,0,1, 0,1,0,1,0,0,1,0,1,1,1]
], dtype=int) % 2
# систематическая форма для (15,4) симплекс-кода

weights = []
for i in range(16):
    u = np.array([int(b) for b in format(i, '04b')])
    c = (u @ G) % 2
    wt = np.sum(c)
    weights.append(wt)

min_d = min(w for w in weights if w > 0)

```